

awk

- Filtro para ficheros de texto programable.
- awk selecciona campos y registros (líneas) de un fichero de entrada y el resultado se muestra en la salida estándar:
- La estructura es:

awk -parámetros *parte_dónde* *programa* nomFich

awk {acción} nomFich

awk -F {acción} nomFich

awk dir1 {acción} nomFich

awk dir1,dir2 {acción} nomFich

awk -f fichPrograma nomFich

awk

- awk considera el fichero de entrada como un conjunto de registros.
 - Por defecto, cada línea es un registro.
- awk divide los registros en campos.
 - Por defecto, cada campo es un conjunto de caracteres separado por el carácter blanco.

- Algunas variables asociadas a campos y registros:

\$0 identifica a todo el registro
\$1 identifica al campo primero
...
\$30 identifica al campo treinta

NR Número del registro
NF Número de campos del registro
FILENAME Nombre del fichero de entrada

awk

- **parte_dónde** indica las direcciones (`dir1 dir2`) donde se aplica la acción del programa.
 - Las direcciones pueden ser expresiones regulares.
 - Las direcciones pueden ser expresiones booleanas.
 - `BEGIN` indica que la acción se ejecute antes de tratar ningún registro (línea)
 - `END` indica que la acción se ejecute después de tratar todos los registros (línea)
- **acción** indica la secuencia de sentencias del programa.
 - Por defecto, cada sentencia en una línea
 - Con `;` pueden ponerse varias sentencias en una misma línea.

awk

- Estructuras más comunes de *parte_dónde programa*

- Caso 1:

```
BEGIN{acción1}  
parte_dónde{acción2}  
END{acción3}
```

- Caso 2:

```
BEGIN{acción1}  
parte_dónde{acción2}
```

- Caso 3:

```
parte_dónde{acción1}  
END{acción2}
```

- Caso 4:

```
parte_dónde{acción}
```

awk

- Sentencias del programa (**acción**) :
 - De escritura:
 - `print` con formato implícito.
 - `printf` con formato explícito (similar al lenguaje C).
 - De asignación.
 - De control de flujo.
 - `if-else`
 - `while`
 - `for ( ; ; )`
 - `for ( in )`
- Si no hay parte **{acción}**, awk ejecuta `{print}`

awk

<code>dir1 {acción}</code>	la acción se aplica al registro (línea) <code>dir1</code>
<code>dir1, dir2 {acción}</code>	la acción se aplica a los registros (líneas) entre <code>dir1</code> y <code>dir2</code>
<code>{acción}</code>	si no se pone dirección (<i>parte_dónde</i>), la acción se aplica a todos los registros (líneas) del fichero
<code>BEGIN {acción}</code>	la acción se aplica antes que a ningún registro (línea)
<code>END {acción}</code>	la acción se aplica después de todos los registros (líneas)
<code>dir1</code>	se aplica <code>print</code> al registro (línea) <code>dir1</code>
<code>dir1, dir2</code>	se aplica <code>print</code> a los registros (líneas) entre <code>dir1</code> y <code>dir2</code>

awk

```
$cd ~/tso/01/palabras
```

```
$awk '{print}' dias
```

```
$awk '{print $0}' dias
```

```
$awk '/^m/{print;print}' dias
```

```
$awk '{print;print}' dias
```

```
$awk '{print "==" ,$1, "--"}' dias
```

```
$awk '/u/,/a/{print;print}' dias
```

```
$awk 'BEGIN{print "pricipio"}
```

```
    /es/{print;print}
```

```
    END{print "final"}' dias
```

awk

```
$cd ~/tso/01/parejas
```

```
$awk '{print $2}' regimenenes
```

```
$awk '{print $2,$2}' regimenenes
```

```
$awk '{print $2,$1}' regimenenes
```

```
$awk '{print $2 $1}' regimenenes
```

```
$awk '{print "comida=", $1, "nombre=", $2}' regimenenes (con 3 comas)
```

```
$awk '{print "comida="$1, "nombre="$2}' regimenenes (con 1 coma)
```

```
$awk '{print "comida="$1"nombre="$2}' regimenenes (sin comas)
```

awk

```
$cd ~/tso/01/parejas
```

```
$awk 'NR>3{print "nombre"$2}' regimen.es
```

```
$awk 'NR==4{print "nombre"$2}' regimen.es
```

```
$awk 'NR<2{print "nombre"$2}' regimen.es
```

```
$awk '/e/{print "lin=",NR, "contenido=", $0}' regimen.es
```

```
$awk '/t/{print NR, "comida=", $1}
```

```
END{print "fich=",FILENAME}' regimen.es
```

awk

```
$cd ~/tso/01/parejas
```

```
$cat fich.awk
```

```
BEGIN{print "muestra los elementos del fichero"}  
{print "reg=", NR, "elem1=", $1, "elem2=", $2}  
END{print "total regs=", NR, "fich=", FILENAME}
```

```
$awk -f fich.awk regimenenes
```

```
$awk -f fich.awk juntas
```

```
$awk -f fich.awk alimentos
```

awk

\$cat fichDatos

```
luis=terol=marcial  
jose=vela=porto  
emilio=gil=martos  
marta=sanz=costa
```

\$ awk -F= 'NR>2{print "seg. apellido:"\$3}' fichDatos

```
seg. apellido:martos  
seg. apellido:costa
```

Expresiones booleanas

- Una expresión es booleana si su resultado es *verdadero* (*true*) o, en caso contrario, es *falso* (*false*).

```
$vi scriptExpBool
```

```
v=4
if test $v -ge 3
then
    echo "$v es Mayor o igual que 3"
else
    echo "$v es Menor que 3"
fi
```

-ge *mayor o igual que*

```
$bash scriptExpBool
```

Expresiones booleanas

- Para la bash, una expresión es booleana si su resultado es *verdadero (true)* o en caso contrario *falso (false)*.

```
$vi scriptCompara
```

```
if test $1 -ge 3
then
    echo "$1 es Mayor o igual que 3"
else
    echo "$1 es Menor que 3"
fi
```

```
$bash scriptCompara 4
```

```
$bash scriptCompara 2
```

Expresiones booleanas

- El resultado de la ejecución de los comandos de la bash puede utilizarse como expresión booleana:
 - Una ejecución con éxito de un comando devuelve **0** (*verdadero/true*).
 - Una ejecución fallida de un comando devuelve un valor distinto de 0 (*falso/false*).

```
$cd ~/tso/01/palabras
```

```
$cp dias dias2
```

```
$echo $?
```

```
0
```

```
$cp noExiste fich2
```

```
$echo $?
```

```
1
```

Expresiones booleanas

scriptPrueba

```
$cd ~/tso/01/palabras  
$vi scriptPrueba  
$bash scriptPrueba
```

```
if cp dias dias2  
then  
    echo "Bien"  
else  
    echo "Mal"  
fi
```

```
$if cp dias dias2 ; then echo "Bien" ; else echo "Mal" ; fi
```

```
$if cp noExiste fich2 ; then echo "Bien" ; else echo "Mal" ; fi
```

Operadores lógicos

- `&&` es el y lógico (and)
- `||` es el o lógico (or)
- `expr1 && expr2` deben ser verdaderas ambas expresiones para que el resultado sea verdadero
- `expr1 || expr2` debe ser verdadera al menos una de las expresiones para que el resultado sea verdadero

Operadores lógicos

- `expr1 && expr2` deben ser verdaderas ambas expresiones para que el resultado sea verdadero

```
$cd ~/tso/01/palabras
```

```
$cp dias dias2 && cp colores cols2
```

```
$echo $?
```

```
0
```

```
$cp dias dias2 && cp noExiste fich2
```

```
$echo $?
```

```
1
```

Operadores lógicos

- `expr1 || expr2` debe ser verdadera al menos una de las expresiones para que el resultado sea verdadero

```
$cd ~/tso/01/palabras
```

```
$cp dias dias2 || cp noExiste fich2
```

```
$echo $?
```

```
0
```

```
$cp ficticio fCopia || cp noExiste fich2
```

```
$echo $?
```

```
1
```

Operadores lógicos

script_AND

```
$cd ~/tso/01/palabras  
$vi script_AND  
$bash script_AND
```

```
if cp dias dias2 && cp colores cols2  
then  
    echo "Bien ambas copias"  
else  
    echo "Mal ambas copias"  
fi
```

Operadores lógicos

script_OR

```
$cd ~/tso/01/palabras
```

```
$vi script_OR
```

```
$bash script_OR
```

```
if cp dias dias2 || cp noExiste fich2
then
    echo "Bien al menos una copia"
else
    echo "Mal ambas copias"
fi
```

Doble evaluación

- La bash ejecuta lo que se encuentra entre ` ` (comillas inversas) y coloca en su lugar la salida estándar del comando.
- `$( )` es equivalente a ` `

```
$cd ~/tso/01
```

```
directorio=`pwd`
```

```
$echo $directorio
```

```
$cat $directorio/palabras/dias
```

```
directorio=$(pwd)
```

Make

- Permite compilar de forma sencilla programas en C:

make [*opciones*] [*objetivo*]

- Estructura fichero Makefile:

```
objetivo1: requisito1 ... requisitoN
```

```
    accion1
```

```
    ...
```

```
    accionN
```

```
objetivo2: requisito1 ... requisitoN
```

```
    accion1
```

```
    ...
```

```
    accionN
```

```
...
```

Make

- Regla para que se ejecuten las acciones:

La fecha del objetivo es anterior a la que tiene al menos uno de los requisitos

- fichero Makefile:

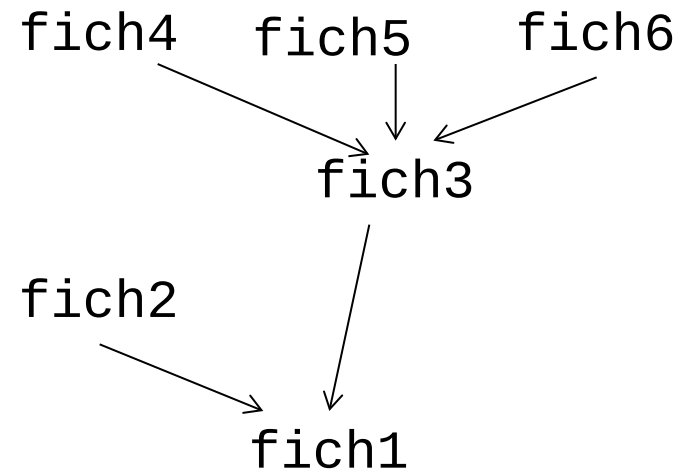
```
obj: req1 req2 req3  
    acc1
```

**acc1 se ejecutará si obj tiene fecha anterior a req1
o a req2 o a req3**

Make

- Dependencias Makefile:

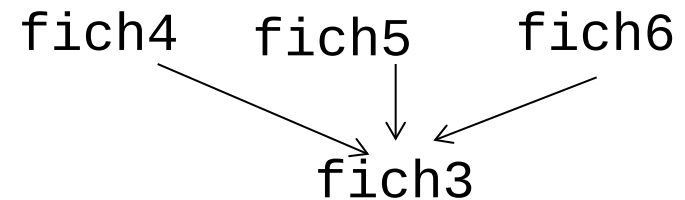
```
fich1: fich2 fich3  
    acc1  
  
fich3: fich4 fich5 fich6  
    acc2
```



Make

- Dependencias Makefile:

```
fich1: fich2 fich3  
    acc1  
  
fich3: fich4 fich5 fich6  
    acc2
```



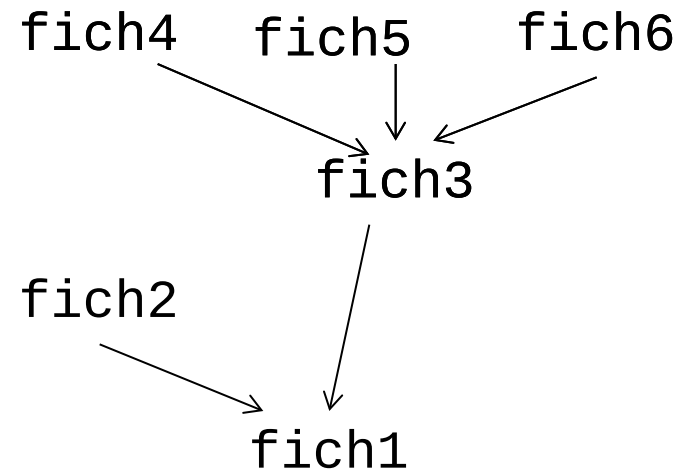
\$make fich3

**acc2 se ejecutará si fich3 (no existe o) tiene
fecha anterior a fich4 o a fich5 o a fich6**

Make

- Dependencias Makefile:

```
fich1: fich2 fich3  
    acc1  
  
fich3: fich4 fich5 fich6  
    acc2
```



\$make fich1

Primero:

acc2 se ejecutará si fich3 (no existe o) tiene fecha anterior a fich4 o a fich5 o a fich6

Después:

acc1 se ejecutará si fich1 (no existe o) tiene fecha anterior a fich2 o a fich3

Make

- Si se invoca al make sin objetivo, es igual que poner el primer objetivo:

```
fich1: fich2 fich3  
      acc1  
  
fich3: fich4 fich5 fich6  
      acc2
```

\$make

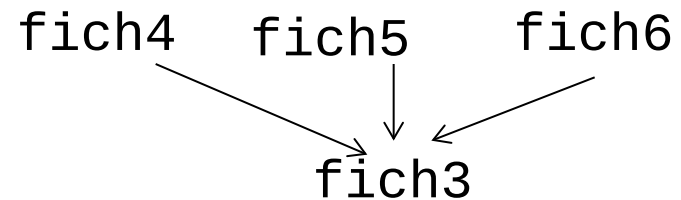
Es igual que

\$make fich1

Make

- La opción **-n** señala las acciones que se ejecutarían (pero no las ejecuta)

```
fich1: fich2 fich3  
      acc2  
  
fich3: fich4 fich5 fich6  
      cp ra rb
```



```
$rm fich3 rb  
$make -n fich3
```

cp ra rb se señalaría como a ejecutar porque **fich3** no existe, pero no se ejecuta. Por lo tanto, no existirá **rb** hasta que no se haga `make fich3`

Make

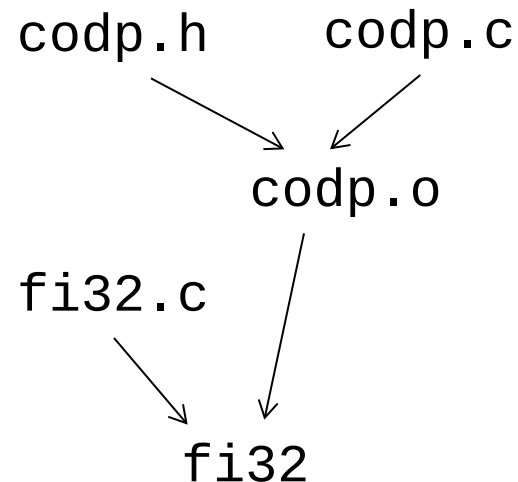
```
$cd ~/tso/01/progs/proy32
```

```
$cat Makefile
```

```
fi32: fi32.c codp.o  
      cc fi32.c -o fi32 codp.o
```

```
codp.o: codp.h codp.c  
        cc -c -o codp.o codp.c
```

```
$make codp.o
```



`cc -c -o codp.o codp.c` se ejecutará si `codp.o` (no existe o) tiene fecha anterior a `codp.h` o a `codp.c`

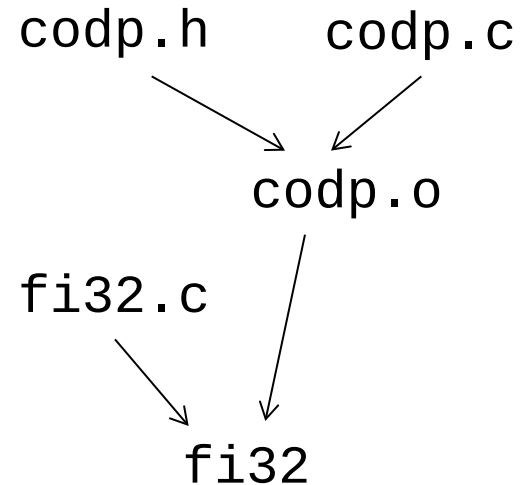
Make

\$cat Makefile

```
fi32: fi32.c codp.o
    cc fi32.c -o fi32 codp.o

codp.o: codp.h codp.c
    cc -c -o codp.o codp.c
```

\$make fi32 | **\$make**



Primero:

`cc -c -o codp.o codp.c` se ejecutará si `codp.o` (no existe o) tiene fecha anterior a `codp.h` o a `codp.c`

Después:

`cc fi32.c -o fi32 codp.o` se ejecutará si `fi32` (no existe o) tiene fecha anterior a `fi32.c` o a `codp.o`